

## Additional Context : CSVs and Commas

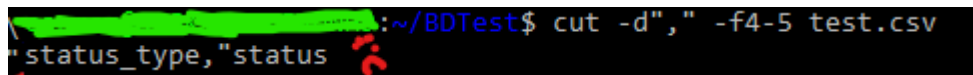
**NB - if you are just looking for an explanation of the code, you can skip past the first section!**

```
#!/bin/bash

cp facebookdata.csv facebookdata-clean.csv
while grep -q '"[^"][*],.*"' facebookdata-clean.csv ;do
    sed -i.bak 's/\("[^"][*]\),\(..*\)\/\1;\2/'
    ↪ facebookdata-clean.csv
done
```

### Why are we doing this...?

Coming from Q6, we know that there is an **issue with the .csv file - the status\_link header**. For csv files to “work” properly in the bash context, we need consistent delimiters - bash is reading these as text files, and won’t do much work in terms of parsing our input. If we try to use cut on our file, we see that the headers aren’t being cut out cleanly - the comma in the middle is being identified as a delimiter:



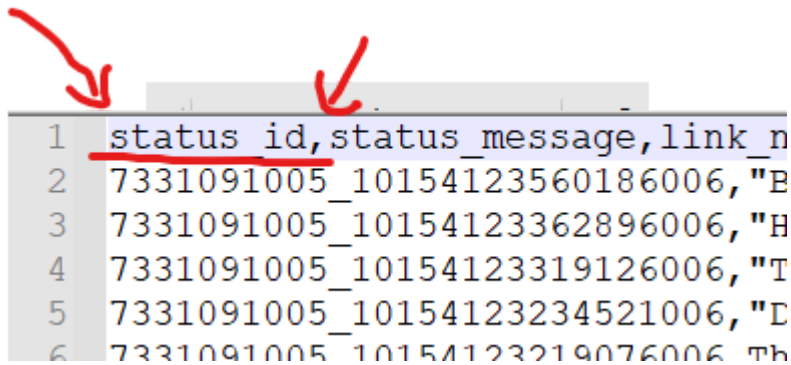
```
~/BDTest$ cut -d"," -f4-5 test.csv
"status_type,"status
```

If you look at the raw text, you’ll see that the status\_link header is wrapped in quotation marks - "status\_link" - this allows programs with more advanced parsers (e.g. Microsoft Excel) to recognise that the comma contained within the quotation marks should not be used as a delimiter. As a result, when handling csv files in bash (and in general) you will often come across this kind of formatting. For a simple example of how this issue can arise, let’s have a look at what happens when I make some changes to a .csv (notably, not an “excel” file like .xlsx) in Excel:

First, let’s look at a cell in excel with no commas:

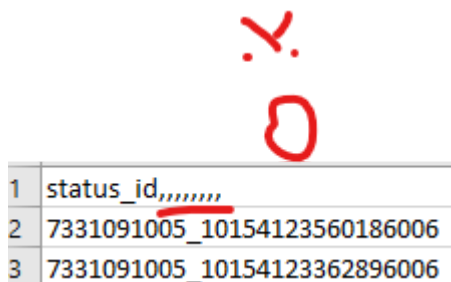
|   | A                          | B       |
|---|----------------------------|---------|
| 1 | status_id                  | status_ |
| 2 | 7331091005_101541235601860 | Ben Sir |
| 3 | 7331091005_101541233628960 | How to  |
| 4 | 7331091005_101541233191260 | The ne  |
| 5 | 7331091005_101541232345210 | Derrick |
| 6 | 7331091005_101541232190760 | The 38  |

status\_id has no comma in it - now let's take a look at the raw text:



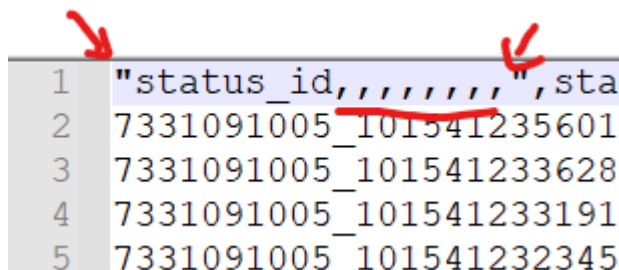
1 status\_id,status\_message,link\_n  
2 7331091005\_10154123560186006,"E  
3 7331091005\_10154123362896006,"H  
4 7331091005\_10154123319126006,"T  
5 7331091005\_10154123234521006,"E  
6 7331091005\_10154123219076006,"Th

We can see that status\_id is followed by a comma that delineates it from status\_message - so far so good. But if we add commas to that cell...



1 status\_id,,,,,,,,,  
2 7331091005\_10154123560186006  
3 7331091005\_10154123362896006

How will excel know that this isn't a cell followed by several empty cells? The answer, as we have noted above, is to wrap it in quotation marks:



1 "status\_id,,,,,,,,,"sta  
2 7331091005\_101541235601  
3 7331091005\_101541233628  
4 7331091005\_101541233191  
5 7331091005\_101541232345

Now Excel (or any software similarly equipped with a decent parser) knows to interpret this as a single cell - not dissimilar to wrapping text in Bash!

As a result of the above, we will often find .csv files in a format that doesn't "suit" bash - so it's important to know how to address this in case we want to use bash for something that a program like excel can't (e.g. editing the first few lines of a 100GB .csv).

## Ok, but what does all this mean?

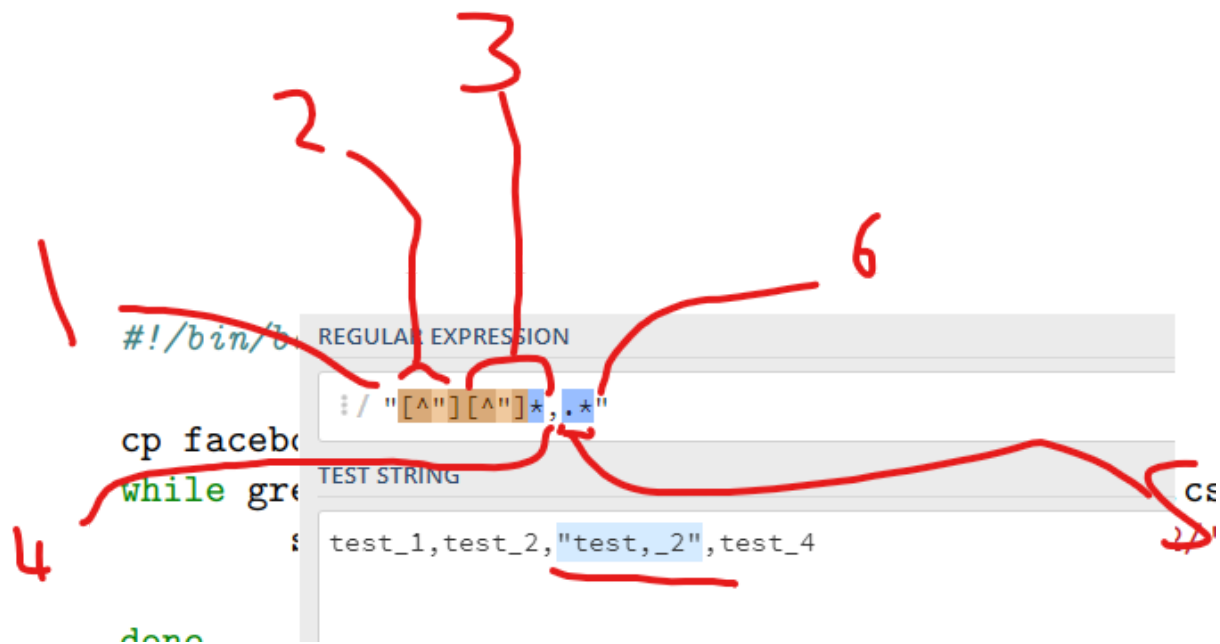
Good question - let's take a look at this piece by piece:

```

1. #!/bin/bash
2 cp facebookdata.csv facebookdata-clean.csv
3 while grep -q '"[^"]*,!*"' facebookdata-clean.csv ;do
    4 sed -i.bak 's/\("[^"]*\)*\),\(..*"\)/\1;\2/'
    ↪ facebookdata-clean.csv
done

```

1. This is the “shebang” - the #! signals that the following text is a direct path to the interpreter - in our case the /bin/bash interpreter.
2. Here, we are simply making a new file - facebookdata-clean.csv - by copying facebookdata.csv. The cp command (more here: <https://man7.org/linux/man-pages/man1/cp.1.html>) takes the form of cp SOURCE DESTINATION. Since we don’t specify directories etc. it’s just going to make a copy of facebookdata.csv in the same folder called facebookdata-clean.csv!
3. Here, we are creating a while loop using a grep find as a condition. The -q (see manual here: [https://linuxcommand.org/lc3\\_man\\_pages/grep1.html](https://linuxcommand.org/lc3_man_pages/grep1.html)) means that grep will not output anything - it just exits with exit code 0 (successful). The while loop will take that “success” code as a reason to keep running (side note: the [[]] or [] syntax you use to evaluate true/false statements works the same way - “true” statements return exit code 0). The facebookdata-clean.csv at the end indicates the target of the grep command.
4. The syntax here (excluding the outer single quotes, which are just encapsulating the text literally) is as follows: match a quote (1), something that isn’t a quote(2), as many non-quote characters as possible - see the star after [^"] (3), a comma(4), any character - see the punctuation mark followed by a star (5), and finally a quotation mark again (6). For a more visual representation, here is the above syntax at work in regex101 (with the above numbers marked). You’ll notice that in the test string area, it is only catching our comma within quotes.



5. Now this is where things get more complicated (particularly because we have to escape a lot of characters in non-extended regex). Essentially, if we are at this line, our grep has found an instance of a comma within quotes that needs addressing, and we are going to use sed to do so. Sed (<https://linux.die.net/man/1/sed>) is one of a few tools we have for this kind of replacement task - we are effectively going to do a “find and replace” with patterns instead of direct text matches (although this is a distinction without a difference - direct text matches are just simpler patterns). **The -i.bak here means “edit in place, create a backup file with .bak extension”.**
6. There is a lot going on here, so we will go piece by piece.

The single quotes are, again, encapsulating the text so that it will be interpreted literally - no special meanings (broadly - in this string a few characters have still been escaped, which I will mention below).

The s at the start flags to sed that we are doing a **substitution operation**. This is followed by a / to indicate the “next clause” (basically sed works on the basis of SOMETHING/SOMETHING/SOMETHING/). Here, we are structuring our command as so: SUBSTITUTION\_FLAG/STUFF\_TO\_FIND/STUFF\_TO\_REPLACE/

This is followed by an escaped parentheses - “\”. What we are doing here is creating a **capture group** - a match that we can refer to later (you’ll see the 1 and 2 near the end - these refer to the first and second capture groups respectively). In this capture group, you’ll notice that the match syntax is the same as the first half of the grep match - “[^"]{1,}” - this matches everything **up until the comma** (see point 4 above for a breakdown). You’ll see that the group, which was opened by an escaped parentheses, is closed by one as well.

Then, outside of any capture group, we have the comma. You may have guessed by now, but what we are doing is capturing the exact same pattern as in the grep string, but doing it in such a way that we can exclude the comma from our replacement.

Following the comma, we have another escaped parentheses - it's capture group 2. Inside here - `(.*)` - we have the rest of the grep match - the `.*` - that matches any number of any character up until a quotation mark. Again, see point 4 above for a more detailed breakdown of this.

Now, we have another clause delimiter (see the `s` flag description above) that tells `sed` that what follows is to be substituted anywhere we found a match for the preceding pattern. This portion is fairly simple - `\1` and `\2` are the escaped symbols for the first and second capture group, and the semicolon in the middle is just a semicolon. So, wherever we had something in the form of `"XXX,YYY"`, we will capture `XXX` as group 1, `YYY` as group 2, and place them in the format of `"XXX;YYY"`, with the comma removed (as we did not capture it!). See below for the direct impact of the command:

```
~/BDTest$ cat test.csv
status_id,status_message,link_name,status_type,"status, link",status_published,num_reactions,num_comments,num_shares,num_likes,num_loves,num_wows,num_hahas,num_sads,num_angrys

~/BDTest$ sed 's/\("[^"]*\)[^"]*\),\("[^"]*\)"/\1;\2/' test.csv
status_id,status_message,link_name,status_type,"status; link",status_published,num_reactions,num_comments,num_shares,num_likes,num_loves,num_wows,num_hahas,num_sads,num_angrys
```

The last `/` is just finishing out the command.

7. The `facebookdata-clean.csv` indicates the target of the `sed` command. I am flagging this separately - the little arrow means that it is on the same line as the `sed` command, but has been moved down due to lack of space!

After all this, if we run our `cut` command on the cleaned up text, we see that we get the correct header fields:

```
~/BDTest$ cut -d"," -f4-5 fixed.csv
status_type,"status; link"
```

Hopefully the above clears things up a little - please note that this is about as difficult a regex as you are likely to see in this course, so do not worry too much if it was difficult to understand on your first pass!